

GZHV2 改进方案（解耦、稳定、轻量）

第一性原则：系统最小单元是“字段”和“图片”，核心流程是“定内容 → 定插图 → 定排版 → 输出”。原则上先分析需求的根源，再从零开始推演出尽量简单的方案。奥卡姆剃刀要求尽量减少假设和冗余逻辑，能删的删，能合并的合并，保持功能足够但不过度设计。

1 目标原则（悬丝剃刀）

- **解耦：**彻底去除固定 8 张卡片的耦合，改为基于字段和图片自由组合的卡片系统。所有内容存储在字段表和图片表，通过卡片表和卡片项表进行组合。
- **稳定：**数据模型、接口和渲染流程保持一致，避免前端和后端之间的状态不一致。以标准 REST API 作为交互契约，所有状态变化都通过接口完成，前端不再维护隐式的状态。
- **轻量：**删除不必要的入口（旧版内容定稿、钩子文案等），合并相似功能。界面采用模块化设计，避免多层嵌套；后端只承担数据和渲染任务，不介入界面逻辑；数据库只保存必要字段。

2 整体架构（轻量分层）

根据上述原则，建议采用三层结构：

层级	主要职责	核心特点
前端 (Vue 3 / Pinia)	研究台、定稿台、排版台等界面模块；提供拖拽式卡片编辑、字段定稿、图片定稿、模板制作等功能	单页应用，组件解耦，状态统一管理；Element Plus 等 UI 库即可满足需求
后端 (Flask)	提供字段和图片 CRUD 接口，卡片组合管理接口，模板管理、渲染和导出接口，ECharts 图表生成等服务	遵循 REST 风格，接口统一；图片生成和渲染由后端完成，不在前端执行 heavy 逻辑
数据库 (SQLite)	持久化研究数据、字段和图片源、卡片编排、模板配置等	模型简单，支持事务；图片和生成图表文件保存在本地文件系统，数据库仅存元信息

该结构能够保证部署简单：单个进程即可运行 Flask 和前端静态文件，SQLite 无需额外维护；必要时可平滑迁移到更强大的数据库。

3 统一数据模型（最小且一致）

为解决当前数据模型混杂的问题，需要统一字段和图片的存储，并通过卡片表组合这些资源。建议的数据表如下：

表名	核心字段	说明
projects	id (PK) , name , created_at 等	每个项目（公司）一条记录，用于隔离不同公司的数据

表名	核心字段	说明
fields	id (PK), project_id (FK), key, label, version_std/business/spread, final_value, status (draft/confirmed), updated_at	研究阶段写入三版本文本，定稿阶段写入 final_value 并标记为 confirmed；取消使用 final_content 表，只使用这一表即可 1
images	id (PK), project_id (FK), asset_key (logo/website_screenshot/chart...), local_path, source_type (source/generated/chart), meta (JSON: 来源参数等), created_at	采集与生成的图片统一存储，asset_key 只表明需求类型，不再绑定卡片序号 2
cards	id (PK), project_id (FK), card_id (稳定 ID, 如 card01), title, sort_order, template_id, created_at	每张卡片的定义，包括顺序、名称、使用的模板等 3
card_items	id (PK), card_id (FK), item_type (field/image), item_key, sort_order, display_role (title/subtitle/body/chart/logo/hero_image/decoration)	表示某张卡片包含的字段或图片，以及它在模板中的角色 3
templates (可选)	id (PK), name, template_json, created_at	用户自定义模板的完整布局 JSON；预置模板也在此表

说明：

1. 所有内容通过 card_items 组合到卡片，而 fields 表保存每个字段的最终文本，避免了 final_content 按卡片编号存储的混乱 1。
2. 图片表不再关联卡片序号，只表明图片属于哪个项目及其类型，这使得图片需求与卡片布局解耦 2。
3. 模板表保存完整的布局及样式信息，模板逻辑由前端或渲染器解析；参数调整保存在 template_json 内。

4 定稿台功能设计（按需求）

为了简化用户流程并符合“定内容→定插图→定排版”的逻辑，可将定稿台拆分为三个清晰的模块：

1. **卡片设置**：新建、删除、排序、启用卡片；为每张卡片勾选需要的字段和图片，并指定显示角色。建议使用拖拽交互：左侧显示所有可用字段和图片，右侧是卡片列表；拖拽条目即可添加到指定卡片。同时可批量自动分配，默认依据字段组别和图片 asset_key 生成一套初始卡片顺序 4。
2. **文字定稿**：按照字段分组列出研究阶段的三版本内容，用户点击版本卡片可一键采用，或在输入区编辑最终文本并保存。保存即更新 fields.final_value 并标记为 confirmed，取消旧版“卡片内容定稿”逻辑 5。
3. **图片定稿**：在定稿台侧边栏集成 image-studio 功能，而非另开窗口。用户选择某个 asset_key，面板展示当前已选图片和候选列表，提供 预览、重新采集、上传/导入、AI 生成 按钮。ECharts 生成的图表在同一面板提供代码调试区域及参数滑块，生成完成后直接将生成图表写入 images 表 6 7。

这样可以避免频繁在不同页面间切换，并且卡片设置与文字/图片定稿界面紧密关联，用户可以随时调整某个卡片的内容和插图。

5 排版台与模板制作

排版台负责将定稿内容渲染到卡片模板，并调节排版细节。改进建议如下：

- 模板选择与参数调节：**排版台左侧提供模板列表（读取自 `templates` 表），用户可快速切换模板；中间区域显示选中卡片的实时渲染结果；右侧参数面板支持调整字体大小、颜色、间距、图片尺寸和位置等⁸。所有参数变更实时更新 `template_json`。
- 模板制作器：**提供跳转入口到高自由度的模板编辑界面，可新建画布、设置背景颜色或图片、添加文本区域和图片区域，设置绑定的 `display_role`，并保存为自定义模板供后续使用。模板编辑也可以基于现有模板复制修改，避免重复工作。
- 实时预览与导出：**排版台支持同时预览多张卡片，保存时将模板 JSON 与卡片组合关系打包，通过后端渲染器生成 HTML/CSS。导出时支持 PNG、PDF 或 PPTX，多种分辨率选项，以满足不同平台的需求。

6 研究与采集管道

研究管道需要与新版模型保持一致：

- 字段采集：**现有研究管道将不同版本的文本写入 `fields` 表的 `standard` / `business` / `spread` 字段；需要新增字段时只需在 `fields` 表插入一条新记录即可，无需修改其他结构。
- 图片采集：**采集逻辑不再尝试为每张固定卡片找一张图片，而是针对 `asset_key` 采集多张候选图片，写入 `images` 表并打分；管道完成后自动选择质量最高的一张设为默认。所有图片均不绑定卡片，由用户在定稿阶段分配。
- 初始卡片生成：**后台可根据字段分组生成默认的 3~5 张卡片配方，作为卡片设置的起点。例如，一张“公司介绍”卡片包含公司名称、简介、成立时间和 Logo，一张“产品与商业模式”卡片包含产品描述和商业模式，再加上一张“竞争格局与总结”卡片。用户可在此基础上增删调整。默认卡片方案写入 `cards` 和 `card_items` 表⁴。

7 接口设计（精简）

所有功能通过 REST 接口暴露，推荐的主要端点如下：

路径	方法	说明
<code>/api/projects</code>	GET/POST	查询、创建项目
<code>/api/fields</code>	GET	按项目返回字段列表与版本内容
<code>/api/fields/<field_id></code>	PATCH	更新字段的 <code>final_value</code> 和 <code>status</code>
<code>/api/images</code>	GET	按项目返回图片槽位及候选情况
<code>/api/images/<asset_key></code>	POST	上传或生成某个槽位的图片
<code>/api/cards</code>	GET/POST/ DELETE	查询、新增、删除卡片
<code>/api/cards/<card_id></code>	PATCH	更新卡片标题、排序、模板等信息
<code>/api/card-items/<card_id></code>	POST/DELETE	批量分配或移除字段/图片到卡片
<code>/api/templates</code>	GET/POST	查询、自定义模板管理

路径	方法	说明
<code>/api/render/<project_id></code>	GET	返回渲染所需的卡片数据和模板，用于排版台预览
<code>/api/export/<project_id></code>	GET	按用户选择的格式导出卡片（PNG/PDF/PPTX）

接口设计保持单一职责，命名清晰，同时避免过多细粒度接口，前端通过一次调用可获取完整的渲染数据。

8 实施要点

- 代码清理：**移除旧版固定 8 张卡片的常量和逻辑，例如 `CARD_TITLES` 与 `CARD_COUNT`⁵，以及遗留的“内容定稿”和“钩子文案”面板⁹。
- 数据迁移：**将历史数据从 `final_content` 迁移到 `fields.final_value` 和 `card_items`。迁移完成后废弃 `final_content` 表。
- 组件重构：**前端拆分为“卡片设置”“文字定稿”“图片定稿”三个面板，使用统一的状态管理；集成 `image-studio` 到当前页面；新增模板制作器组件。
- 渲染器更新：**调整 `render-data-loader.js` 与 `template-renderer.js`，从新的接口加载数据并渲染；不再加载 legacy 1-8 卡片回退路径¹⁰。
- 测试与反馈：**围绕核心流程编写单元测试和端到端测试，验证字段更新、图片生成和卡片渲染；邀请典型用户体验新版流程并收集反馈。

9 示意图

下面的示意图展示了改进后的整体流程、数据模型和界面模块：



图中各部分对应本文提出的改进：顶层是“定内容→定插图→定排版→输出”的核心流程；左下角是统一的数据模型；右侧是卡片设置、文字定稿、图片定稿的界面示例；排版台和模板制作位于中下；最后是导出与 API 设计。通过这些改进，系统达到解耦、稳定、轻量的目标。

1 2 6 7 10 raw.githubusercontent.com

<https://raw.githubusercontent.com/zbwnrepyd/GZHv2/main/docs/architecture.md>

3 4 raw.githubusercontent.com

https://raw.githubusercontent.com/zbwnrepyd/GZHv2/main/db/init_composition_db.sql

5 9 raw.githubusercontent.com

<https://raw.githubusercontent.com/zbwnrepyd/GZHv2/main/webapp/static/js/editor.js>

8 raw.githubusercontent.com

<https://raw.githubusercontent.com/zbwnrepyd/GZHv2/main/canvas/js/template-renderer.js>